

Ein GPU-beschleunigter Restarted-PDHG-Solver für diskretes Optimal Transport

Felix Summerer

28. August 2026

Zusammenfassung

Diskrete Optimal-Transport-Probleme (OT) lassen sich als lineare Programme in Kantorovich-Formulierung schreiben, deren Größe jedoch quadratisch mit der Auflösung der zugrundeliegenden Bilder wächst und generische LP-Solver damit schnell an Speicher- und Laufzeitgrenzen bringt. Diese Arbeit stellt einen GPU-beschleunigten Restarted Primal-Dual Hybrid Gradient (rPDHG) Solver für diskretes OT vor, der die spezielle Struktur der Transport-Nebenbedingungen ausnutzt, um sowohl die Matrix-Vektor-Produkte als auch die Spektralnorm der Restriktionsmatrix in geschlossener Form zu berechnen. Der Solver kombiniert eine Pock–Chambolle-Vorkonditionierung, eine Malitsky–Pock-Liniensuche sowie eine adaptive Restart-Strategie nach Applegate et al. (2021), die auf einem skaleninvarianten Fortschrittsmaß basiert. Die Implementierung ist speicherfrei im inneren Iterationskern (alle Operationen erfolgen in-place auf der GPU via CuPy) und wird auf 400 DOTmark-Probleminstanzen (alle 10 Kategorien, 32×32) evaluiert und gegen den kommerziellen Solver Gurobi verglichen. Die Ergebnisse zeigen, dass der Solver robust gegen die Wahl mehrerer Hyperparameter ist und Zielwerte findet, die im Median auf 3.1×10^{-9} relativ genau mit Gurobis Referenzlösung übereinstimmen; in absoluter Laufzeit bleibt er auf den getesteten, vergleichsweise kleinen Probleminstanzen mit median $4.65\times$ jedoch noch deutlich hinter Gurobi zurück.

1 Einleitung

Das Optimal-Transport-Problem (OT) sucht den kostengünstigsten Weg, eine Massenverteilung in eine andere zu überführen, und ist seit L. Kantorovichs Relaxierung des ursprünglich von Monge formulierten Problems ein zentrales Werkzeug in Statistik, Bildverarbeitung und maschinellem Lernen. In seiner diskreten Form – etwa beim Vergleich zweier Bilder als Verteilungen über ihre Pixel – lässt sich OT exakt als lineares Programm formulieren. Der Nachteil dieser Exaktheit ist die Größe des resultierenden LPs: Für zwei Bilder mit je $N = H \cdot W$ Pixeln besitzt die Transportvariable N^2 Einträge, was klassische Simplex- oder Innere-Punkte-Solver bereits bei moderaten Bildauflösungen an ihre Grenzen bringt.

Ziel dieser Arbeit ist es, einen First-Order-Solver zu entwickeln, der diese Skalierungsprobleme durch (a) Ausnutzung der speziellen Struktur der Transport-Restriktionsmatrix und (b) vollständige Ausführung auf der GPU adressiert. Der gewählte algorithmische Rahmen ist der *restarted Primal-Dual Hybrid Gradient* (rPDHG) Algorithmus, wie er in ähnlicher Form im Open-Source-Solver PDLP von Google Research eingesetzt wird [1]. Der Beitrag dieser Arbeit ist eine auf das OT-Zwei-Nachbarn-Muster der Restriktionsmatrix zugeschnittene, speicherfreie GPU-Implementierung inklusive geschlossener Spektralnormberechnung, Pock–Chambolle-Vorkonditionierung, Malitsky–Pock-Liniensuche und adaptivem Restart, sowie deren empirische Evaluation auf dem DOTmark-Benchmark [4] im Vergleich zu Gurobi.

Der Rest dieses Berichts ist wie folgt gegliedert: Abschnitt 2 führt die mathematischen Grundlagen ein (Kantorovich-Formulierung, PDHG, Dualitätslücke). Abschnitt 3 beschreibt die Restart- und Vorkonditionierungsstrategien sowie die konkrete GPU-Implementierung. Abschnitt 4 beschreibt den experimentellen Aufbau, Abschnitt 5 präsentiert die Ergebnisse, und Abschnitt 6 fasst zusammen und skizziert offene Fragen.

2 Grundlagen

2.1 Diskretes Optimal Transport

Wir betrachten die diskrete Kantorovich-Formulierung

$$\min_{x \in \mathbb{R}^{N^2}} c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad (1)$$

bei der eine erste N -dimensionale Massenverteilung $a \in \mathbb{R}^N$ in eine zweite Verteilung $b \in \mathbb{R}^N$ überführt wird. Dabei ist $N = H \cdot W$ die Anzahl der Pixel eines $H \times W$ -Bildes, und wir betrachten das vollständig dichte Transportproblem, bei dem jeder Pixel des einen Bildes potenziell Masse zu jedem Pixel des anderen transportiert.

Die Transportvariablen x_{ij} werden zu einem Vektor $x \in \mathbb{R}^{N^2}$ mit Index $k = i \cdot N + j$ linearisiert. Die Restriktionsmatrix $A \in \mathbb{R}^{2N \times N^2}$ kodiert die Massenerhaltung: jede Spalte von A besitzt genau zwei von Null verschiedene Einträge (beide gleich 1), die der zugehörigen Angebots-beziehungsweise Nachfragegleichung entsprechen. Diese sehr dünn besetzte, regelmäßige Struktur ist der Ausgangspunkt für alle im Folgenden beschriebenen Optimierungen.

2.2 Spektralnorm der Restriktionsmatrix

Die Schrittweiten des PDHG-Algorithmus hängen von der Spektralnorm $\|A\|_2$ ab, die üblicherweise per Potenziteration geschätzt wird. Für die OT-Restriktionsmatrix lässt sich $\|A\|_2$ jedoch exakt angeben: Es gilt $\|A\|_2^2 = \lambda_{\max}(AA^\top)$, und aufgrund der Struktur der Transport-Nebenbedingungen zerfällt $AA^\top$ in ein Blockschema

$$AA^\top = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} NI_N & J_N \\ J_N & NI_N \end{pmatrix}, \quad (2)$$

wobei I_N die Einheitsmatrix und J_N die Matrix aus lauter Einsen ist. Eine Eigenwertanalyse dieser Blockstruktur zeigt, dass der maximale Eigenwert exakt $2N$ beträgt, sodass

$$\|A\|_2 = \sqrt{2N} \quad (3)$$

in geschlossener Form gilt. Dies ersetzt die approximative Potenziteration durch eine exakte, konstante Berechnung und erfüllt die theoretischen Anforderungen an die PDHG-Schrittweitenwahl streng.

2.3 Primal-Dual Hybrid Gradient (PDHG)

Der PDHG-Algorithmus basiert auf dem Lagrangian

$$\mathcal{L}(x, y) = c^\top x + \langle y, b - Ax \rangle, \quad (4)$$

mit Dualvariablen $y \in \mathbb{R}^{2N}$, wobei die Indikatorfunktion $\iota_{\mathbb{R}_{\geq 0}^N}(x)$ implizit die Nichtnegativitätsbedingung $x \geq 0$ erzwingt und über die Projektion $P_{\geq 0}(x_k) = \max(0, x_k)$ realisiert wird.

Als primäres Konvergenzmaß verwenden wir die *Dualitätslücke* (Duality Gap), die die Differenz zwischen primälem Zielwert $f(x) = c^\top x$ und dualem Zielwert $g(y) = \inf_x \mathcal{L}(x, y)$ misst:

$$\text{Gap}(x, y) = c^\top x - \inf_{x'} \mathcal{L}(x', y). \quad (5)$$

Am Optimum schließt sich diese Lücke auf null; sie erlaubt es, Stagnation zu erkennen und adaptive Restarts auszulösen (Abbildung 1).

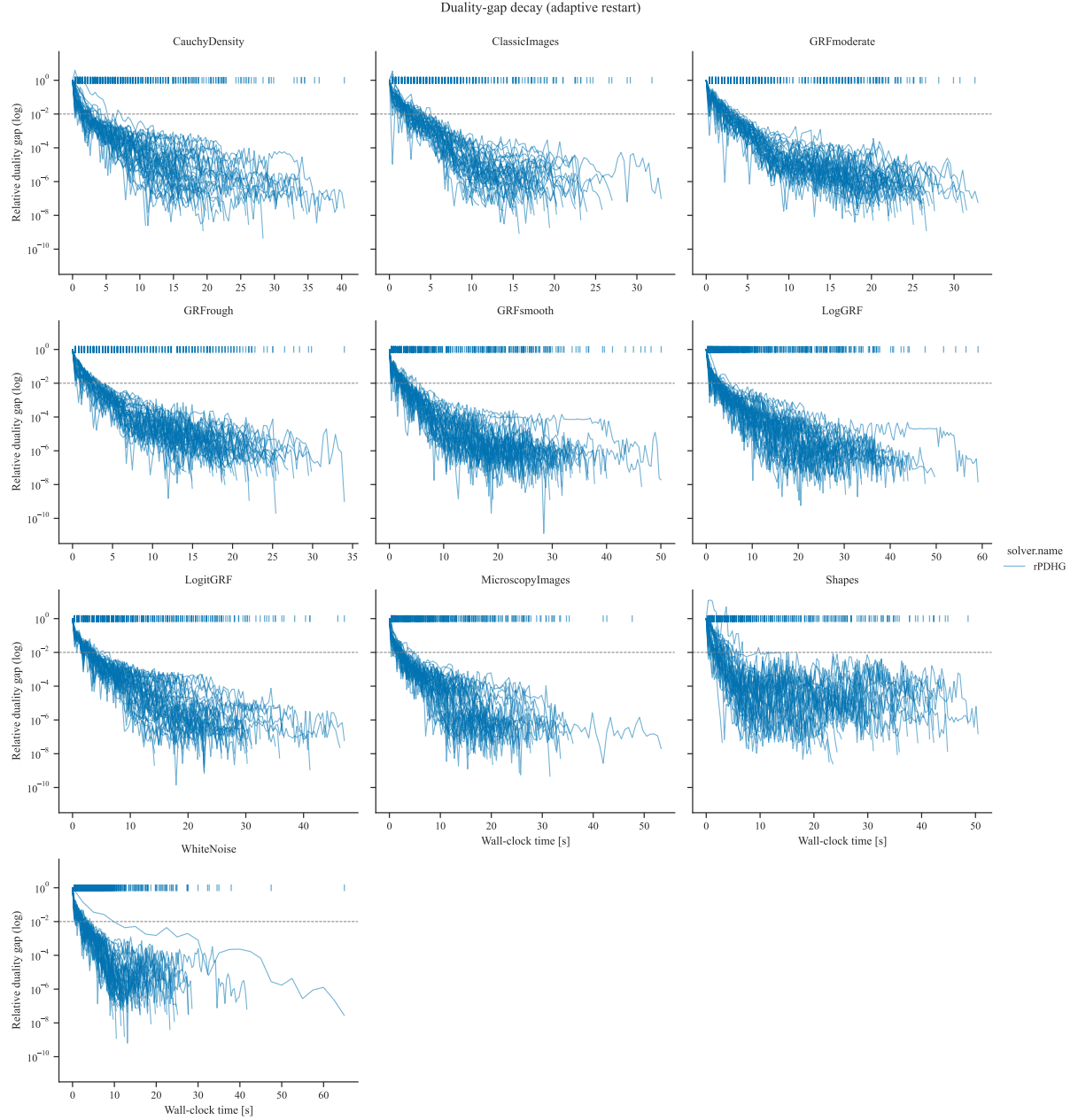


Abbildung 1: Verlauf der relativen Dualitätslücke über die Wall-Clock-Zeit, je ein Panel pro DOTmark-Klasse (alle 400 Instanzen des in Abschnitt 4 beschriebenen 32×32 -Benchmarks, Kurven pro Instanz). Vertikale Striche am oberen Rand markieren Restart-Zeitpunkte; die gestrichelte Linie markiert die relative Toleranz 10^{-2} .

3 Methodik

3.1 Restart-Strategien

Der Standard-PDHG-Algorithmus neigt insbesondere bei nicht strikt konvexen Problemen zu oszillierendem Verhalten. Um Konvergenz zu beschleunigen und Stagnation zu vermeiden, implementieren wir zwei Restart-Strategien, die Primäl- und Dualvariablen (beziehungsweise deren Momentum) zurücksetzen.

Fixe Restarts. Hierbei wird strikt alle K Iterationen zurückgesetzt, $k \equiv 0 \pmod{K}$. Dieser Ansatz ist rechnerisch günstig und einfach zu implementieren, jedoch hängt die optimale Frequenz K stark vom jeweiligen Problem ab.

Adaptive Restarts. Inspiriert von Applegate et al. (2021) [1] verwenden wir ein normalisiertes Fortschrittsmaß μ , das die Dualitätslücke mit primärer und dualer Infeasibilität kombiniert, normalisiert auf die seit dem letzten Restart zurückgelegte Distanz:

$$\mu(z, z_{\text{ref}}) = \frac{\text{Gap}(z)}{\text{dist}(z, z_{\text{ref}})} + \|r_p\|_2 + \|r_d\|_2, \quad (6)$$

wobei $z = (x, y)$ das aktuelle Iterat und z_{ref} der Startpunkt der aktuellen Epoche (das heißt des Intervalls seit dem letzten Restart) ist. Die Distanz wird in einer gewichteten euklidischen Norm gemessen, um den Skalenunterschied zwischen Primäl- und Dualvariablen auszugleichen:

$$\text{dist}(z, z_{\text{ref}}) = \sqrt{\omega \|x - x_{\text{ref}}\|_2^2 + \frac{1}{\omega} \|y - y_{\text{ref}}\|_2^2}, \quad \omega = \sqrt{\tau/\sigma}. \quad (7)$$

An jedem Diagnoseschritt wird μ sowohl für das aktuelle Iterat z_{curr} als auch für den ergodischen Mittelwert z_{avg} ausgewertet; das Minimum μ_{best} bestimmt den Restart-Kandidaten z^* . Sofern die aktuelle Epoche mindestens `min_epoch_length` Iterationen gelaufen ist, wird ein Restart ausgelöst, wenn

$$\begin{aligned} \mu_{\text{best}} &\leq 0.9 \cdot \mu_{\text{prev_restart}} && \text{(hinreichender Fortschritt), oder} && (8) \\ 0.1 \mu_{\text{prev_restart}} &\geq \mu_{\text{best}} > \mu_{\text{last_check}} && \text{(Stagnation nach starkem Fortschritt).} && (9) \end{aligned}$$

Unabhängig davon erzwingt eine harte Obergrenze `fixed_iter_restart` einen Restart, sobald die Epoche diese Länge erreicht, um im ungünstigsten Fall die Epochenlänge zu beschränken. Durch die Normierung der KKT-Residuen auf die zurückgelegte Distanz erhält man ein skaleninvariantes Maß für „Fortschritt pro Bewegungseinheit“, das reines Oszillieren um ein Optimum von echtem Fortschritt unterscheidet.

3.2 Vorkonditionierung nach Pock–Chambolle

Die Schrittweiten τ, σ werden mittels einer an die OT-Struktur angepassten Pock–Chambolle-Vorkonditionierung [2] bestimmt. Da jede Spalte von A genau zwei und jede Zeile genau N Nichtnulleinträge besitzt, ergeben sich Basiswerte

$$\tau_{\text{base}} = 2^{-\alpha}, \quad \sigma_{\text{base}} = N^{-(2-\alpha)}. \quad (10)$$

Um primäre und duale Skalen auszugleichen, wird das Datenverhältnis $\omega = \|c\|_2/\|b\|_2$ symmetrisch eingerechnet:

$$\tau = \tau_{\text{base}} \sqrt{\omega} \cdot s, \quad \sigma = \frac{\sigma_{\text{base}}}{\sqrt{\omega}} \cdot s, \quad (11)$$

mit Sicherheitsfaktor $s = \text{`safety\_margin`}$ (Standardwert 0.999), der $\tau\sigma\|A\|_2^2 < 1$ strikt garantiert – die hinreichende Bedingung für Konvergenz des klassischen PDHG. Mit der geschlossenen Spektralnorm (3) folgt

$$\tau\sigma\|A\|_2^2 = 2^{1-\alpha}N^{\alpha-1}s^2 = \left(\frac{2}{N}\right)^{1-\alpha}s^2, \quad (12)$$

was beim Standardwert $\alpha = 1$ exakt s^2 ergibt. Für $\alpha \neq 1$ verschiebt der Exponent das Produkt (das heißt α tauscht Primäl- gegen Dualschrittweite), Konvergenz bleibt aber garantiert, solange das Produkt unter 1 bleibt.

3.3 Liniensuche

Um trotz aggressiven Schrittweitenwachstums (Faktor θ , Standardwert 1.05, angewendet nach jedem „sauberen“ – das heißt ohne Schrumpfung akzeptierten – Schritt) die Konvergenzbedingung nicht zu verletzen, wird vor jedem Schritt eine Malitsky–Pock-artige Akzeptanzbedingung [3] geprüft:

$$2|\langle \Delta y, A \Delta x \rangle| \leq 0.95 \left(\frac{1}{\tau} \|\Delta x\|^2 + \frac{1}{\sigma} \|\Delta y\|^2 \right), \quad (13)$$

mit $\Delta x = x_{k+1} - x_k$, $\Delta y = y_{k+1} - y_k$. Schlägt der Test fehl, werden τ, σ um den Faktor `step_shrinkage` (Standardwert 0.75) reduziert – jedoch nicht unter einen Boden von 10^{-6} des Werts bei Schleifeneintritt – und der Schritt wiederholt (maximal 10 Versuche, danach wird die Vorkonditionierung neu berechnet und ein sicherer Schritt ausgeführt).

3.4 GPU-Implementierung ohne Matrixmaterialisierung

Da jede Spalte von A genau zwei Einträge besitzt (Angebotsindex i , Nachfrageindex $N + j$), lassen sich alle benötigten Matrix-Vektor-Produkte ohne Instanziierung von A berechnen:

- $(A^\top y)_{ij} = y_i + y_{N+j}$ wird als Broadcast-Summe `y[:N, None] + y[None, N:]` berechnet.
- $(A\tilde{x})_i$ für $i < N$ ist die Zeilensumme von $\tilde{x}$ (als $N \times N$ -Matrix aufgefasst), für $i \geq N$ die Spaltensumme.

Ein voller PDHG-Schritt

$$\begin{aligned} x_{k+1} &= P_{\geq 0}(x_k - \tau(c - A^\top y_k)), \\ y_{k+1} &= y_k + \sigma(b - A(2x_{k+1} - x_k)), \end{aligned} \quad (14)$$

wird so ausschließlich mit Broadcast- und Reduktionsoperationen auf CuPy realisiert. Von den Bestandteilen der äußeren Iterationsschleife ist bislang nur der Liniensuchen-Akzeptanztest vollständig allokatonsfrei: die Differenzen $\Delta x, \Delta y$ sowie die dafür benötigten Zeilen-/Spaltensummen werden in einmalig vorallozierte Zwischenpuffer geschrieben (`out=-Argument`), und die Iteratenfortschreibung geschieht durch Vertauschen von Python-Referenzen statt durch Kopieren. Zwei weitere Bestandteile allozieren dagegen noch, unterscheiden sich aber deutlich in der Frequenz: Die Welford-Mittelung des ergodischen Iterats ($x_{\text{avg}} += (x_{k+1} - x_{\text{avg}})/k$, in der Implementierung äquivalent über `cp.add(x_avg, x_next * ik, out=x_avg)` berechnet) alloziert das Zwischenprodukt `x_next * ik` mangels `out=-Argument` bei jeder Iteration neu; die Restart-Buchhaltung (`.copy()` der Restart-Kandidaten und Referenzpunkte) alloziert nur an Diagnose-Checkpoints beziehungsweise bei tatsächlichen Restarts, also deutlich seltener. Der `one_pdhg_step_gpu_inplace`-Kern selbst (Angebots-/Nachfrage-Zeilen-/Spaltensummen, siehe `onestep.py`) alloziert ebenfalls pro Schritt einige kleine temporäre Arrays der Größe N . Eine vollständig allokatonsfreie Fassung sowohl dieses Kerns als auch der Welford-Mittelung ist als Erweiterung denkbar (in beiden Fällen genügt ein zusätzlicher, einmalig vorallozierter Scratch-Puffer der Größe N beziehungsweise N^2), war für die hier getesteten Problemistanzgrößen aber nicht laufzeitbestimmend. Der teure diagnostische Block (volle KKT-Residuen, Gap-Auswertung, Restart-Entscheidung) wird nur alle `diagnostik_i` Iterationen ausgeführt und alloziert – unabhängig von der obigen Diskussion – ebenfalls temporäre Arrays, da er ohnehin nur einen kleinen Bruchteil der Iterationen betrifft.

3.5 Grenzen der GPU-Auslastung: Host-Device-Synchronisation

Obwohl sämtliche Arrays des Solvers auf der GPU liegen und der innere Kern keine Heap-Allokationen vornimmt, bleibt die Kontrollflusslogik der äußeren Iterationsschleife *datenabhängig*: sowohl die Akzeptanzentscheidung der Liniensuche (13) als auch die Restart- und Abbruchentscheidungen im diagnostischen Block müssen von einer Python-`if`-Anweisung ausgewertet werden. CuPy führt Arithmetik auf `cp.ndarray`-Objekten asynchron aus; sobald ein Ergebnis jedoch in einem Python-`if`, `max()/min()` oder `float(...)` verwendet wird, muss CuPy es zu einem konkreten Python-Wert auflösen, was einen impliziten `cudaStreamSynchronize` erzwingt – eine harte Barriere, die die gesamte bis dahin aufgebaute Kernel-Warteschlange abarbeiten lässt, bevor Python fortfährt. Das passiert mindestens einmal pro Iteration (Liniensuchen-Akzeptanztest, wiederholt bei jedem der bis zu zehn Verwerfungsversuche) und zusätzlich rund zehnfach in jeder Diagnose-Iteration (zwei Metrikauswertungen für z_{curr} und z_{avg} mit je fünf `float(...)`-Konvertierungen, plus Vergleiche im adaptiven Restart-Modus).

Für die getesteten DOTmark-Größen benötigt ein einzelner elementweiser oder reduzierender Kernel auf der getesteten GTX 1050 Ti nur wenige bis einige Dutzend Mikrosekunden – eine Größenordnung, die mit dem `cudaStreamSynchronize`- und CuPy-Dispatch-Overhead selbst vergleichbar ist. Da die Schleife pro Iteration mindestens einen, in Diagnose-Iterationen bis zu elf solcher Syncs erzwingt, leert die GPU wiederholt ihre Kernel-Warteschlange und bleibt anschließend untätig. Das Resultat sollte eine stark „spikige“ Auslastung mit niedrigem Duty-Cycle statt einer kontinuierlichen Last sein.

Empirische Absicherung. Ein sauberer Rerun des `diagnostik_i`-Sweeps (alle 45 LogGRF-Paare bei 32×32 , `diagnostik_i` $\in \{10, 25, 50\}$) bestätigt dies indirekt (Abbildung 2): Die *Gesamtlaufzeit* zeigt erwartungsgemäß *keinen* sauberen Trend – ein größeres Diagnoseintervall erkennt Konvergenz später, was den Sync-Vorteil teilweise wieder aufhebt –, aber die Metrik *Sekunden pro Iteration*, die diesen Overshoot-Effekt herausrechnet, zeigt den erwarteten Effekt sauber: im gepaarten Vergleich über alle 45 Instanzen ist sie bei `diagnostik_i`=10 auf **45 von 45 Instanzen** höher als bei `diagnostik_i`=50 (Median 12.5 %, Mittelwert 14.1 % Reduktion) – eine robuste, vom Konvergenz-Overshoot-Effekt bereinigte Bestätigung des Synchronisationskosten-Arguments.

Ein parallel aufgezeichnetes GPU-Auslastungsprofil (`nvidia-smi`, $\approx 25\text{--}30$ Hz Abtastrate) zeigt dagegen keinen sichtbaren Einbruch und liegt konstant bei 85–87 % (Abbildung 3). Das widerspricht dem Argument nicht, sondern zeigt eine Auflösungsgrenze des Werkzeugs: `nvidia-smi` meldet nur, ob *irgendein* Kernel innerhalb seines Abtastfensters aktiv war – bei Kernelaufrufen im Mikrosekundenabstand liegt dieses Fenster um Größenordnungen über den postulierten Synchronisationslücken. Eine direkte Auflösung erfordert ein Werkzeug mit Kernel-Zeitachsen-Auflösung (Nsight Systems) und bleibt offen; die Sekunden-pro-Iteration-Messung bleibt daher die eigentliche empirische Stütze dieses Abschnitts.

Dieser Effekt ist strukturell, kein einmaliges Implementierungsversehen: Jedes restart-basierte First-Order-Verfahren mit datenabhängiger Liniensuch- oder Restart-Entscheidung pro Iteration muss diese auf dem Host auflösen. Eine Beseitigung erfordert, die Entscheidungslogik selbst auf das Gerät zu verlagern – etwa eine branchfreie Formulierung via `cp.where` statt Python-`if`, CUDA-Graph-Capture der Iterationsschleife, oder das Batchen mehrerer unabhängiger Instanzen zu einem größeren elementweisen Problem, damit sich die fixen Synchronisationskosten pro Iteration über deutlich mehr tatsächliche GPU-Arbeit amortisieren.

4 Experimenteller Aufbau

Der Solver wird auf den DOTmark-Benchmarkproblemen [4] evaluiert, einem Standard-Datensatz für diskretes Optimal Transport bestehend aus Bildpaaren verschiedener Kategorien (unter anderem CauchyDensity, ClassicImages) in Auflösungen von 32×32 und 64×64 Pixeln. Als Referenz

Runtime vs. diagnostic-block frequency (host-device sync cost)

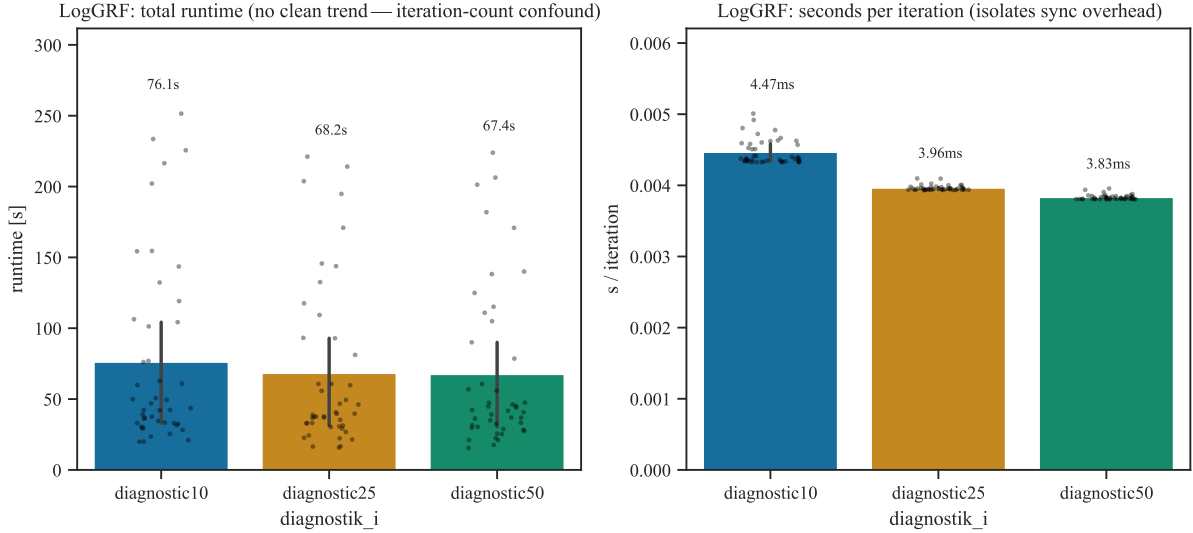


Abbildung 2: LogGRF, 32×32 , $n = 45$ Instanzpaare je **diagnostik_i**-Variante. Links: Gesamtlaufzeit (kein sauberer Trend). Rechts: Sekunden pro Iteration (isoliert die Synchronisationskosten pro Schritt vom Konvergenz-Overshoot-Effekt).

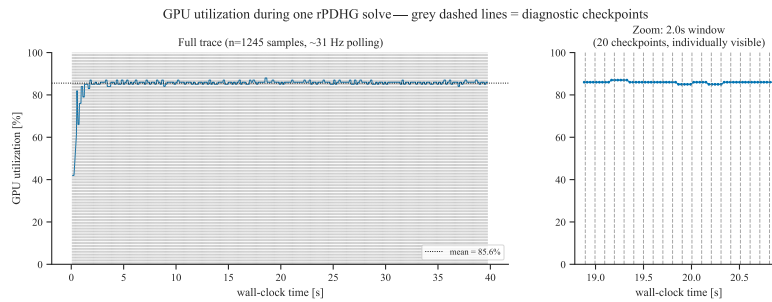


Abbildung 3: GPU-Auslastung über die Zeit während eines repräsentativen rPDHG-Laufs (CauchyDensity, 32×32 , **diagnostik_i** = 25); das rechte Panel zoomt auf ein 2-Sekunden-Fenster, in dem einzelne Diagnose-Checkpoints (graue gestrichelte Linien) unterscheidbar werden. Die im linken Panel durchgehend hohe, kaum schwankende Auslastung ist ein Auflösungsartefakt von **nvidia-smi** (siehe Text).

dient der kommerzielle LP-Solver Gurobi (Version 13.0), der die Kantorovich-Formulierung (1) direkt löst.

Alle Experimente wurden auf einem Rechner mit AMD Ryzen 6-Kern-CPU (3.59 GHz), 16 GB RAM und einer NVIDIA GeForce GTX 1050 Ti GPU (4 GB VRAM, Treiber 560.94) unter Windows 10 durchgeführt, mit Python 3.14, NumPy 2.4 und SciPy 1.16. Neben dem direkten Laufzeitvergleich mit Gurobi wurde die Sensitivität des Solvers gegenüber zwei Hyperparametern untersucht: `min_epoch_length` (Mindestanzahl an Iterationen vor einem adaptiven Restart) und `rebalancing_threshold` (Schwellenwert, ab dem das Verhältnis von primärem zu dualen Residuum ein Nachjustieren von τ und σ auslöst).

Hauptbenchmark: 400 Instanzen bei 32×32 . Für den direkten Vergleich mit Gurobi (Abschnitt 5) wurde je DOTmark-Kategorie eine Auswahl von 40 der $\binom{10}{2} = 45$ möglichen Bildpaare der ersten zehn Instanzen bei 32×32 Pixeln gelöst, insgesamt 400 Instanzen, jede mit beiden Solvern. rPDHG lief mit den in Abschnitt 3 beschriebenen Standardparametern ($\theta = 1.05$, $\alpha = 1$, `step_shrinkage` = 0.75, `rebalancing_threshold` = 1.5, `min_epoch_length` = 100, Toleranzen 10^{-8}), Gurobi mit denselben relativen Toleranzen (`FeasibilityTol`, `OptimalityTol`, `BarConvTol` = 10^{-8}) und automatischem Crossover. Um CUDA-Kontextinitialisierung und CuPy-JIT-Kompilierung nicht in die gemessene Laufzeit einfließen zu lassen, wird vor der ersten getakteten rPDHG-Ausführung ein einmaliger Aufwärmloop auf einer kleinen synthetischen Instanz durchgeführt; zudem wird vor und nach jedem getakteten Solver-Aufruf explizit auf den GPU-Stream synchronisiert (`cudaStreamSynchronize`), damit asynchron nachlaufende CuPy-Kernel nicht in die Messung des *nächsten* Laufs durchsickern. Alle 400 rPDHG- und alle 400 Gurobi-Läufe erreichten den Status `completed`.

5 Ergebnisse

5.1 Sensitivität gegenüber sekundären Hyperparametern

Zusätzlich zur Hauptkonfiguration wurde die Sensitivität des Solvers gegenüber zwei sekundären Hyperparametern untersucht: `min_epoch_length` (Mindestanzahl an Iterationen vor einem adaptiven Restart) und der Rebalancing-Schwelle `rebalancing_threshold` (Abbildungen 4 und 5). In beiden Fällen sind die Laufzeiten über die getesteten Werte hinweg praktisch nicht unterscheidbar – der Solver ist bezüglich dieser beiden Parameter robust und benötigt kein aufwändiges Tuning.

5.2 Vergleich mit Gurobi

Abbildung 6 vergleicht die Laufzeit des rPDHG-Solvers mit Gurobi getrennt je DOTmark-Klasse (400 Instanzen, siehe Abschnitt 4); Abbildung 7 zeigt denselben Vergleich als Streudiagramm mit beiden Achsen im Log-Maßstab, wobei jeder Punkt eine Problem Instanz ist. Tabelle 1 fasst die Medianlaufzeiten und deren Verhältnis je Klasse zusammen.

Über alle 400 Instanzen hinweg ist rPDHG im Median $4.65\times$ (Mittelwert $5.15\times$) langsamer als Gurobi und unterbietet Gurobi nur auf einer einzigen Instanz (0.25 %) – dieser Median wird instanzweise über alle 400 Läufe gebildet und liegt daher etwas unter dem einfachen Mittel der zehn Klassen-Mediane aus Tabelle 1 ($\approx 4.84\times$). Das Verhältnis schwankt je Klasse zwischen etwa $3.1\times$ (ClassicImages) und $7.6\times$ (Shapes) – Gurobi übertrifft den rPDHG-Solver auf allen getesteten Klassen deutlich in der absoluten Laufzeit.

5.3 Korrektheit der Lösung

Ein Laufzeitvorteil für Gurobi wäre irrelevant, wenn rPDHG dabei eine andere – schlechtere – Lösung findet. Abbildung 8 zeigt daher für jede der 400 Instanzen die relative Diffe-

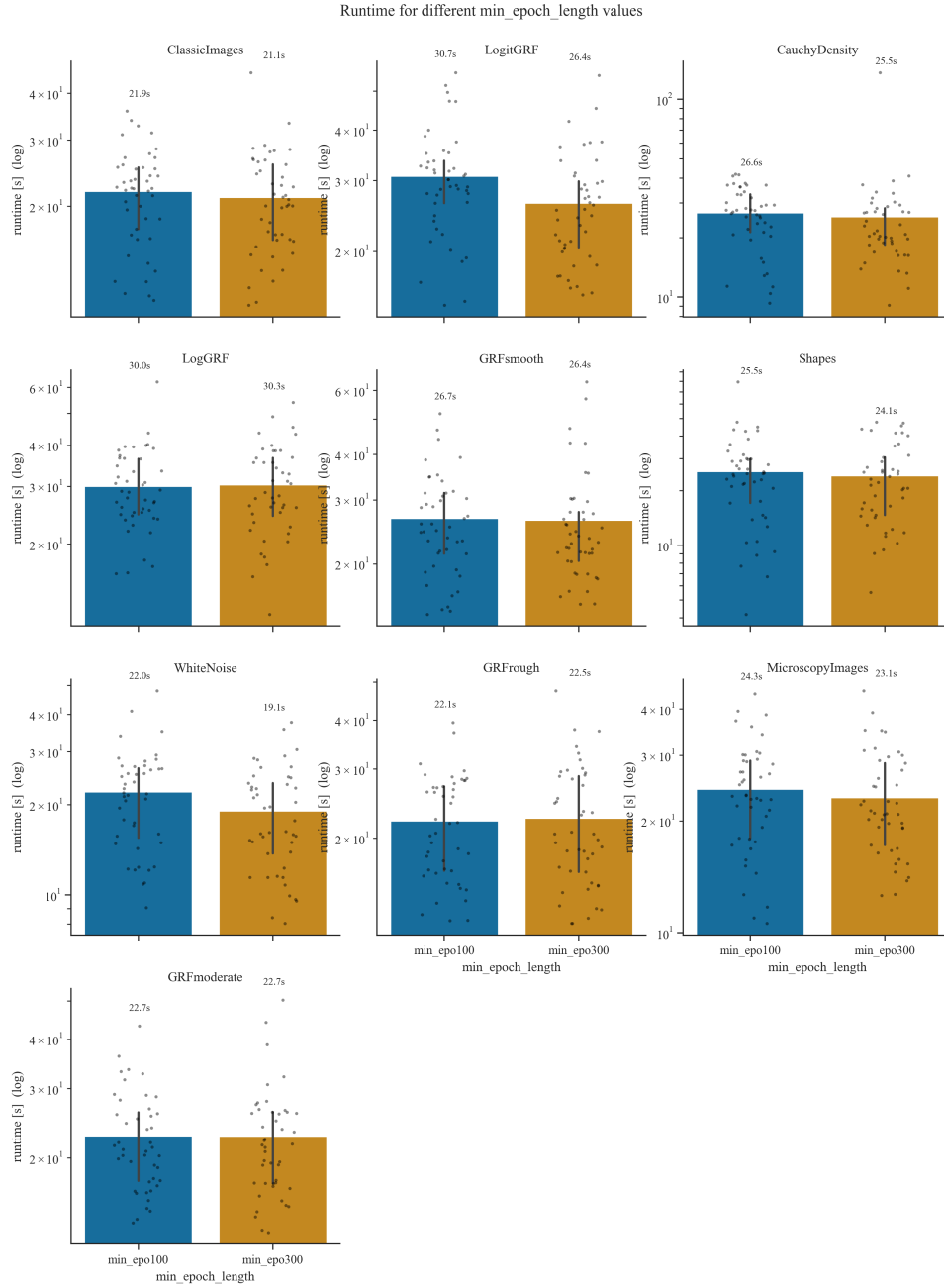


Abbildung 4: Laufzeit für verschiedene Werte von min_epoch_length.

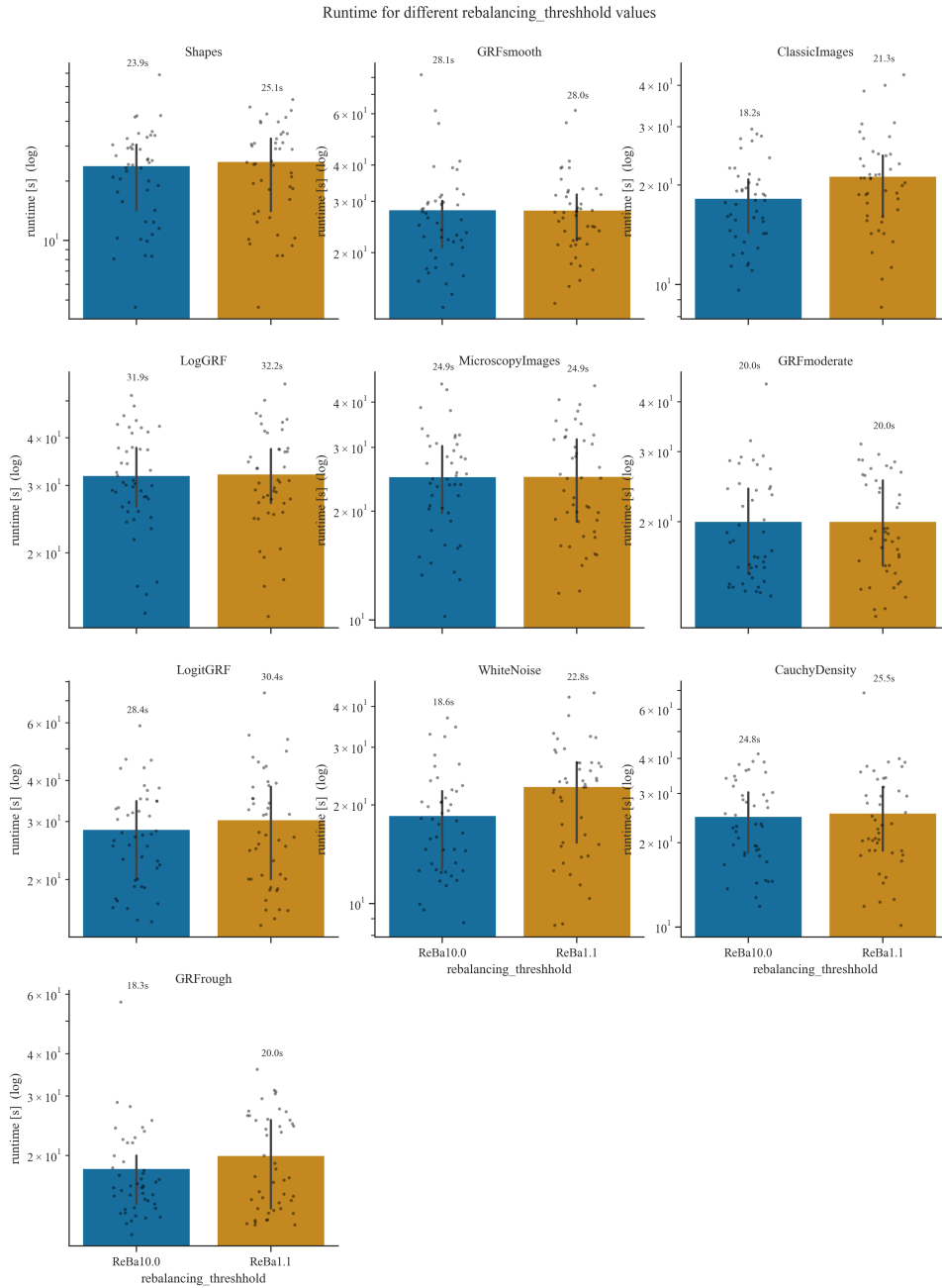


Abbildung 5: Laufzeit für verschiedene Werte der Rebalancing-Schwelle `rebalancing_threshold`.

Runtime per solver, per DOTmark class

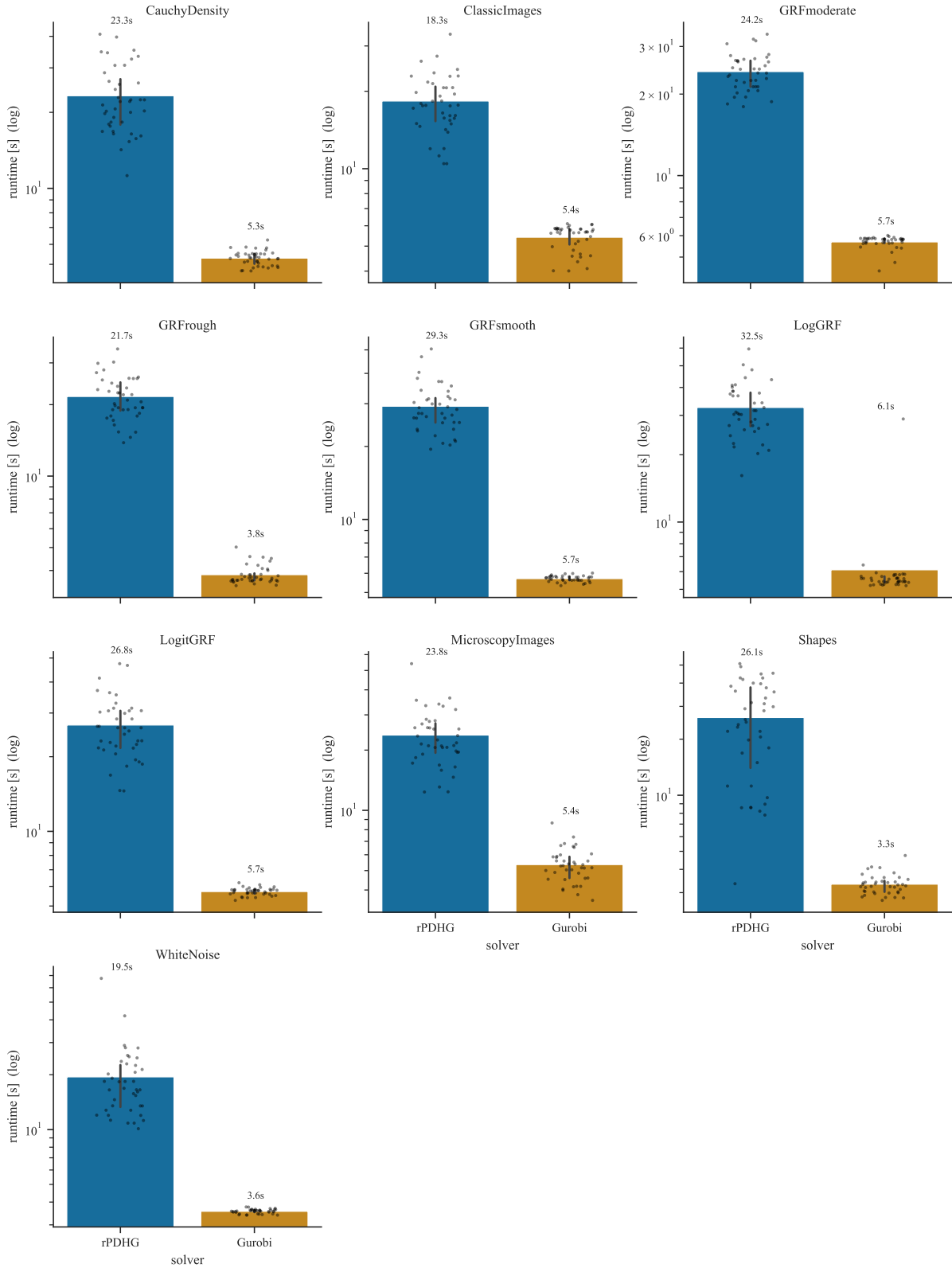


Abbildung 6: Laufzeit von rPDHG und Gurobi je DOTmark-Klasse (Balken = Mittelwert, Fehlerbalken = 50%-Perzentilintervall, $n = 40$ Instanzen je Klasse). Tabelle 1 zeigt zum Vergleich die robusteren Mediane derselben Daten; die beiden Statistiken weichen wegen einzelner langsamer Ausreißer leicht voneinander ab.

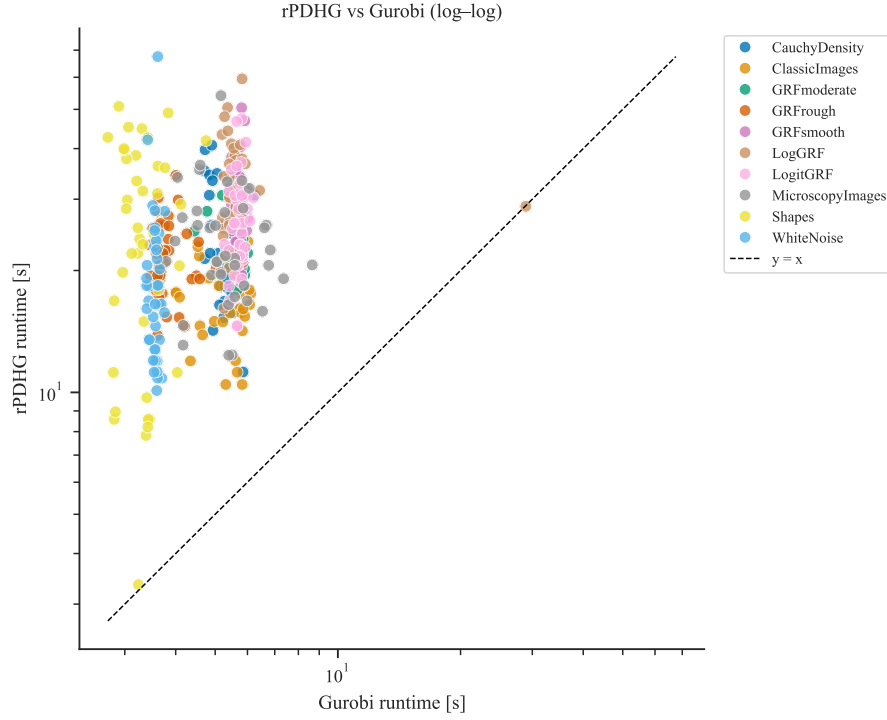


Abbildung 7: Laufzeit von rPDHG gegen Gurobi, Log-Log, je Probleminstance ($n = 400$). Die gestrichelte Linie $y = x$ markiert Gleichlauf; Punkte darunter wären rPDHG-Siege.

Tabelle 1: Median-Laufzeit je DOTmark-Klasse ($n = 40$ Instanzen je Zeile) und Verhältnis rPDHG/Gurobi.

DOTmark-Klasse	rPDHG [s]	Gurobi [s]	Verhältnis
CauchyDensity	21.73	5.31	$4.09\times$
ClassicImages	17.64	5.67	$3.11\times$
GRFmoderate	23.93	5.81	$4.12\times$
GRFrough	20.73	3.70	$5.61\times$
GRFsmooth	27.89	5.73	$4.87\times$
LogGRF	30.85	5.46	$5.65\times$
LogitGRF	26.16	5.75	$4.55\times$
MicroscopyImages	21.60	5.29	$4.09\times$
Shapes	25.08	3.28	$7.64\times$
WhiteNoise	16.68	3.57	$4.68\times$

renz zwischen dem primären Zielwert von rPDHG und dem von Gurobi (als Referenzlösung), $|f_{\text{rPDHG}} - f_{\text{Gurobi}}|/(1 + |f_{\text{Gurobi}}|)$.

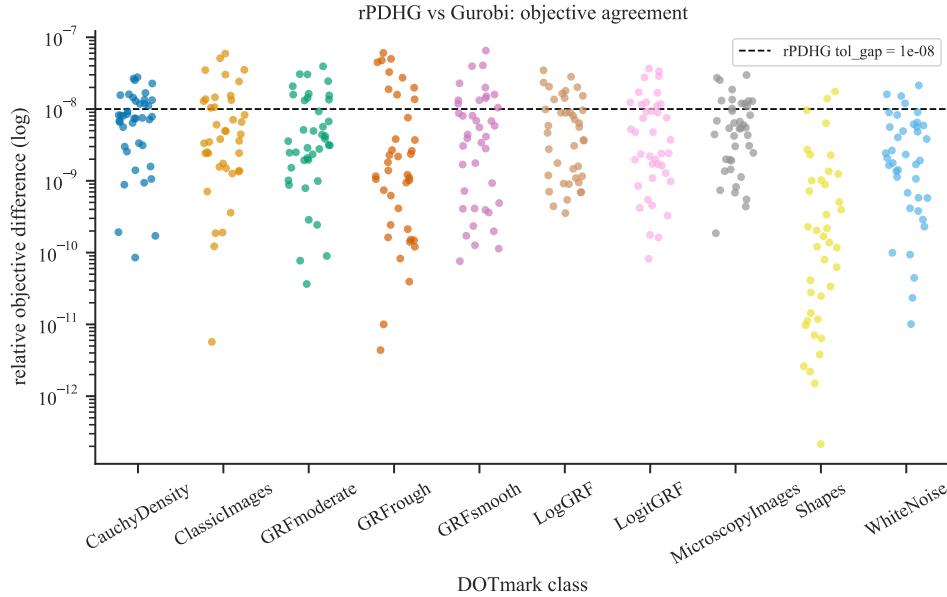


Abbildung 8: Relative Differenz des primären Zielwerts von rPDHG gegenüber Gurobi je DOTmark-Klasse. Die gestrichelte Linie markiert rPDHGs eigene Zielgenauigkeit `tol_gap = 10-8`.

Der Median der relativen Zielwertdifferenz liegt bei 3.1×10^{-9} , der Mittelwert bei 7.4×10^{-9} ; auf 99 der 400 Instanzen liegt die Differenz knapp über rPDHGs eigener Toleranz 10^{-8} , im schlechtesten Fall bei 6.5×10^{-8} – also weiterhin um Größenordnungen unterhalb praktischer relevanter Fehler. rPDHG findet damit durchgehend Zielwerte, die mit der von Gurobi berechneten Referenzlösung übereinstimmen; der in Abschnitt 5.2 beobachtete Laufzeitrückstand geht nicht auf eine geringere Lösungsgüte zurück.

5.4 Performance-Profil

Ergänzend zu den klassenweisen Mittelwerten zeigt Abbildung 9 ein Dolan–Moré-Performance-Profil [5] über alle 400 Instanzen: Für einen Faktor τ gibt die Kurve den Anteil der Probleme an, auf denen der jeweilige Solver höchstens τ -mal langsamer als der auf dieser Instanz schnellste Solver ist.

Gurobi liegt bei $\tau = 1$ bereits bei nahezu 100 %, ist also auf praktisch jeder Instanz der schnellste Solver – konsistent mit dem einen rPDHG-Sieg aus Abschnitt 5.2. Die rPDHG-Kurve steigt erst bei $\tau \approx 3$ –4 merklich an und erreicht erst bei $\tau \approx 10$ nahezu alle Instanzen, was den in Tabelle 1 beobachteten Bereich von $3\times$ bis $8\times$ widerspiegelt.

5.5 Restart-Verhalten

Abbildung 10 schlüsselt die im Median ≈ 17 Restarts pro Lauf (rPDHG-Iterationen im Median 6101, siehe auch Tabelle 1 für die Laufzeiten) danach auf, wie viele Iterationen vor dem ersten, zwischen aufeinanderfolgenden und nach dem letzten Restart vergehen.

Auf sieben der zehn Klassen macht die Phase nach dem letzten Restart den größten Anteil der Gesamtiterationen aus – der Solver verbringt dort also den größten Teil der Laufzeit in einer finalen, restart-freien Polierphase, nicht in wiederholten, kurzen Epochen. Bei den restlichen drei Klassen (GRFsmooth, LogGRF, Shapes) ist stattdessen die mittlere Phase *zwischen* zwei Restarts im Schnitt am längsten, die Polierphase danach vergleichbar kurz – diese Klassen scheinen

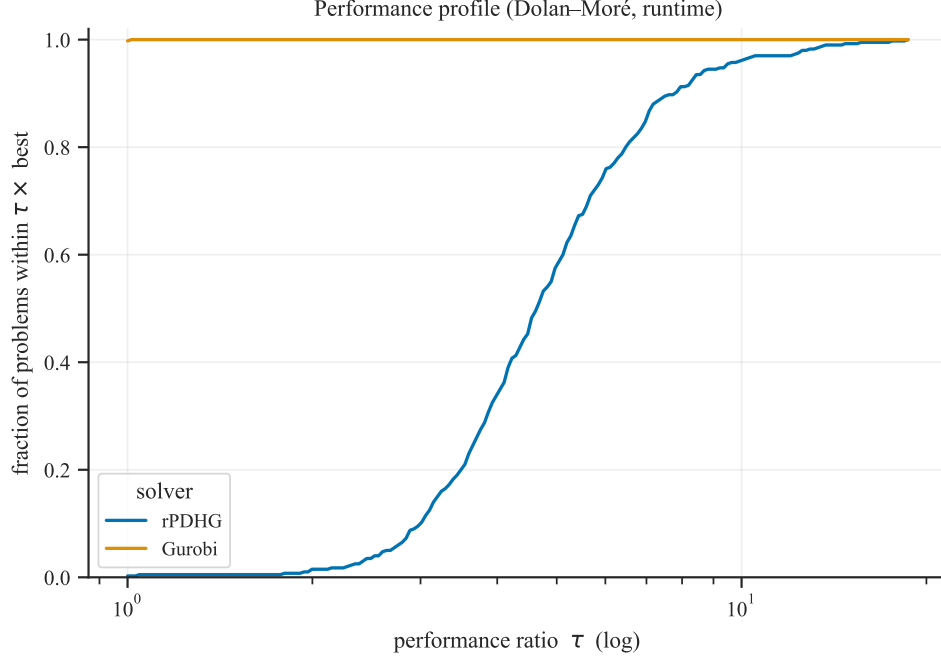


Abbildung 9: Performance-Profil (Dolan–Moré) über alle 400 Instanzen des 32×32 -Benchmarks.

also bis kurz vor Erreichen der Zieltoleranz noch produktive Restarts auszulösen, statt früh in eine restart-freie Endphase überzugehen. Insgesamt ist dies konsistent mit dem in Abbildung 1 sichtbaren Muster: frühe, dichte Restarts während der anfänglichen schnellen Gap-Reduktion, gefolgt von einer längeren Phase feinerer Konvergenz mit selteneren oder keinen weiteren Restarts.

6 Diskussion und Fazit

Diese Arbeit hat einen GPU-beschleunigten Restarted-PDHG-Solver für diskretes Optimal Transport vorgestellt, der die Zwei-Nachbarn-Struktur der Transport-Restriktionsmatrix ausnutzt, um sowohl die Spektralnorm in geschlossener Form zu berechnen als auch alle Matrix-Vektor-Produkte ohne Materialisierung von A auszuführen – kombiniert mit Pock–Chambolle-Vorkonditionierung, Malitsky–Pock-Liniensuche und einem adaptiven, skaleninvarianten Restart-Kriterium nach Applegate et al. (2021).

Der Solver ist robust gegenüber der Feinjustierung sekundärer Hyperparameter (`min_epoch_length`, `rebalancing_threshold`) und erreicht auf 400 DOTmark-Instanzen (32×32) Zielwerte, die im Median auf 3.1×10^{-9} mit Gurobis Referenzlösung übereinstimmen (Abschnitt 5.2) – der Geschwindigkeitsvergleich wird also nicht durch Genauigkeitsunterschiede verzerrt. In absoluter Laufzeit bleibt rPDHG auf diesen kleinen Instanzen dennoch deutlich hinter Gurobi zurück (Median $4.65\times$ langsamer), maßgeblich wegen des in Abschnitt 3.5 beschriebenen Host-Device-Synchronisationsengpasses: Dessen annähernd konstante Kosten pro Iteration fallen bei kleinen N unverhältnismäßig stark ins Gewicht, während die eigentliche Rechenarbeit nur mit N^2 wächst. Der Vorteil des linear statt kubisch skalierenden First-Order-Ansatzes dürfte sich daher erst bei deutlich größeren Instanzen zeigen – ob dem so ist, bleibt offen.

Offene Fragen, die sich aus den Ergebnissen dieser Arbeit ergeben, sind unter anderem: die Beseitigung des Synchronisationsengpasses (zum Beispiel durch eine branchfreie Liniensuche oder CUDA-Graph-Capture), Skalierungsexperimente jenseits von 64×64 (zum Beispiel 128×128), eine Ablation fixer gegenüber adaptiver Restart-Strategien anhand der vorliegenden Trajektorien sowie eine formale Konvergenzanalyse des kombinierten Liniensuche-/Restart-Schemas.

Restart cadence: iterations spent before / between / after restarts

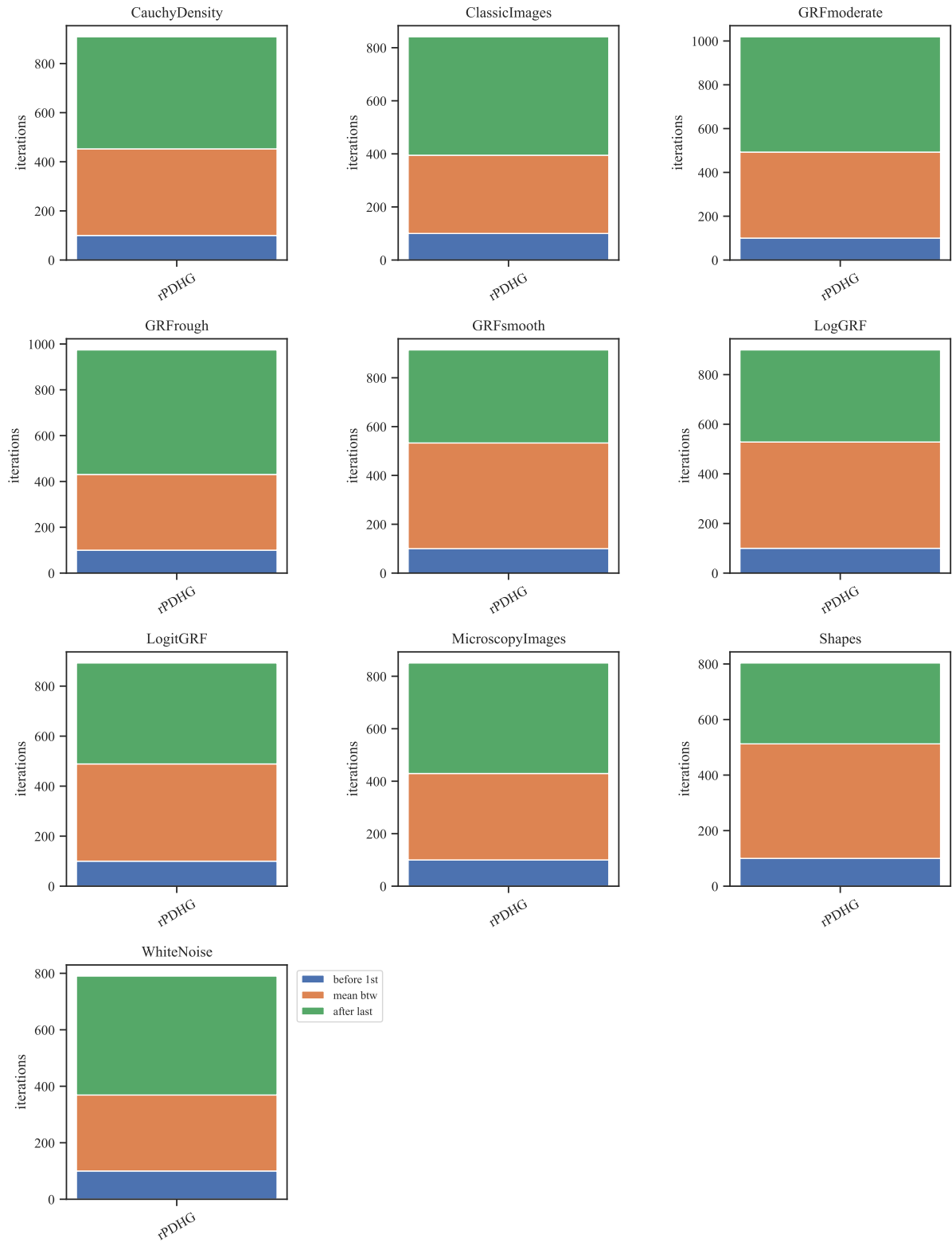


Abbildung 10: Iterationen vor dem ersten, im Mittel zwischen aufeinanderfolgenden, und nach dem letzten Restart, je DOTmark-Klasse (rPDHG, $n = 40$ je Klasse).

Literatur

- [1] D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O’Donoghue, W. Schudy. *Practical Large-Scale Linear Programming using Primal-Dual Hybrid Gradient*. Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [2] A. Chambolle, T. Pock. *A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging*. Journal of Mathematical Imaging and Vision, 40(1):120–145, 2011.
- [3] Y. Malitsky, T. Pock. *A First-Order Primal-Dual Algorithm with Linesearch*. SIAM Journal on Optimization, 28(1):411–432, 2018.
- [4] J. Schrieber, D. Schuhmacher, C. Gottschlich. *DOTmark – A Benchmark for Discrete Optimal Transport*. IEEE Access, 5:271–282, 2016.
- [5] E. D. Dolan, J. J. Moré. *Benchmarking Optimization Software with Performance Profiles*. Mathematical Programming, 91(2):201–213, 2002.

A Reflexion zum Einsatz von KI-Werkzeugen

Für dieses Projekt habe ich Claude (Anthropic), Gemini (Google) und ChatGPT (OpenAI) eingesetzt.

Bereiche mit wesentlichem KI-Einsatz. Ausformulieren der README.md, Erstellen der Grafiken und Schreiben des Abschlussberichts. Am Code beschränkte sich der KI-Einsatz auf Prototyping einzelner Bausteine (vor allem Auswertungs- und Visualisierungsskripte) sowie punktuelle, von mir angestoßene Änderungsvorschläge an bereits bestehendem Code – nicht auf den Kern des Solvers.

Wo KI besonders hilfreich war. Beim Erstellen der Grafiken und des Abschlussberichts wurde durch KI erhebliche Zeitersparnis erzielt.

Eigenständige Arbeit und manuelle Überarbeitung. Lesen diverser Basisliteratur zur Projektplanung. Wesentliche Planung des Ablaufs der Experimente. Die Kernimplementierung des Algorithmus (`algorithms/pdhg_restarted_cu.py`, `onestep.py`) sowie das iterative Testen, Anpassen und Verbessern bis hin zum finalen Projektstand habe ich selbst durchgeführt.

Alle KI-Vorschläge am Algorithmus wurden von mir inhaltlich geprüft, bevor sie übernommen wurden; die Verantwortung für Richtigkeit und Nachvollziehbarkeit der Aussagen liegt bei mir.